



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 04

Objective & Lecture Mapping: In previous labs, you explored Uninformed Search strategies like BFS and UCS inside `agent.py` [cite: 1]. Today, we transition to **Informed Search**. You will enhance your agent by implementing the **A* Search** algorithm, using **Heuristics** to drastically reduce the number of explored nodes in the `visual_grid_game.py` environment [cite: 4]. You will start with basic heuristic functions and connect them to your search logic.

Part 1: Practical Implementation — Building the A* Agent

Step 1.1: Implementing the Heuristic Functions

Informed search algorithms require a heuristic function, $h(n)$, which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called `Week_04` in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class [cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula $h(n) = |x_1 - x_2| + |y_1 - y_2|$ and returns the integer value.
4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. You will need to `import math` for this.
5. *Testing Checkpoint:* Print the outputs of both functions for a mock start position (0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.0.

Step 1.2: Implementing A* Search

A* evaluates nodes by combining the path cost so far, $g(n)$, and the estimated cost to the goal, $h(n)$. The evaluation function is $f(n) = g(n) + h(n)$.

1. Inside `SearchAgent`, create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan')`.
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states`.
3. Push the initial state into the priority queue. Unlike UCS which only uses $g(n)$, your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)`. For the starting node, $g(n) = 0$. Calculate $h(n)$ using your chosen heuristic method, and set $f(n) = g(n) + h(n)$.
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost`. If `current_pos == goal_pos`, return the `path_taken`. Otherwise, add `current_pos` to `reached_states`.
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate $g_{\text{new}} = g_{\text{current}} + 1$, h_{new} using your heuristic, and $f_{\text{new}} = g_{\text{new}} + h_{\text{new}}$. Push the new tuple to the priority queue.

Step 1.3: Integrating A* into the Agent's Decision Loop

Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent`.
2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos`. Call your `astar_search` method and store the returned list of actions in `self.plan`.

5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()`. Run the simulation and observe the optimized pathfinding!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 05, complete the following analytical questions.

1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?

UCS orders the frontier purely by $g(n)$ the actual cost accumulated so far from the start. It has no idea where the goal is, so it expands outward uniformly in every direction, like a growing circle centered on the start. A* orders by $f(n) = g(n) + h(n)$ it adds an *estimate* of the remaining cost to the goal, so the search is actively pulled in the goal's direction rather than expanding blindly. In fact, UCS is just the special case of A* where $h(n) = 0$ for every node.

2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?

On a 4-way grid where every move costs exactly 1 and only changes x or y by 1, Manhattan distance ($|dx| + |dy|$) is exactly the number of moves needed in the best case with no walls in the way. Walls can only make the real path longer, never shorter so Manhattan distance can never overestimate the true remaining cost. That's the definition of admissible: $h(n) \leq \text{true cost}$, always.

If the heuristic were not admissible (it overestimates somewhere), A* loses its optimality guarantee. A node on the true shortest path could get an inflated $f(n)$ that makes it look worse than a longer, suboptimal path, so A* could pop the goal via the wrong route and return a path that isn't actually shortest the algorithm still terminates, it just stops being trustworthy.

3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?

No, not as-is. With diagonal movement, one diagonal step covers both a row and column change in a single move, but Manhattan distance would still count that as 2 (one for x, one for y) so it overestimates the true cost whenever diagonal moves are useful. That makes it inadmissible under 8-way movement. You'd switch to:

- Chebyshev distance ($\max(|dx|, |dy|)$) if diagonal moves cost the same as straight moves (1 per step), or
- Euclidean/octile distance if diagonal moves cost $\sqrt{2}$ (true geometric distance) octile distance specifically ($\max(|dx|, |dy|) + (\sqrt{2}-1)\min(|dx|, |dy|)$) is the standard admissible choice when diagonal cost is $\sqrt{2}$.

4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.

Nearest-food-only is weak because it's myopic it ignores that after eating the closest pellet, the agent still has to travel to every other one, potentially backtracking across the whole map. A stronger, still-admissible option: $h(n) = \text{distance to the nearest remaining food} + \text{MST cost over all remaining food positions}$. The MST (minimum spanning tree) over the food points gives a lower bound on the cost of visiting all of them once you're near the cluster, so combined with the "get to the cluster first" term, it never overestimates the true remaining cost but is far more informative than single-nearest-food. (This is the classic approach used in Pacman-style multi-goal search assignments worth naming that if your write-up wants a citation-style answer.)

Once Completed Get a PDF of the completed File and Push into the Week 04 branch in the Github Project

Finalize and Lock Lab 04 Documentation



FACULTY OF COMPUTING